Introduction to Cache Coherence

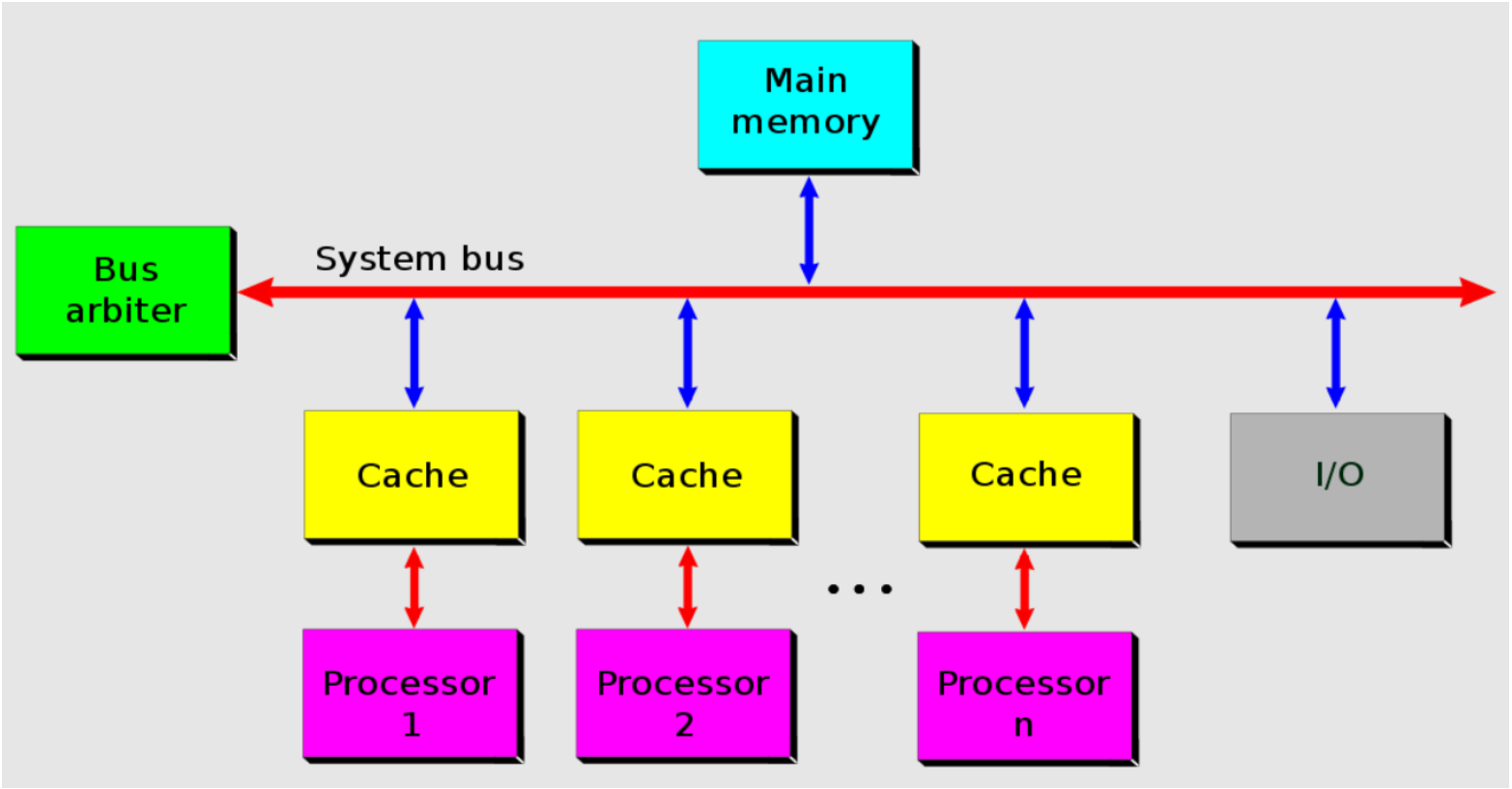
John Jose

Associate Professor

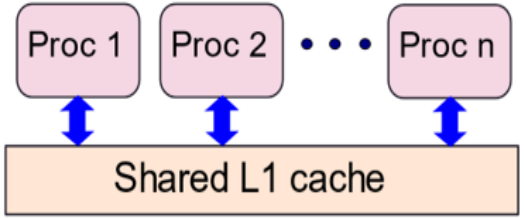
Department of Computer Science & Engineering

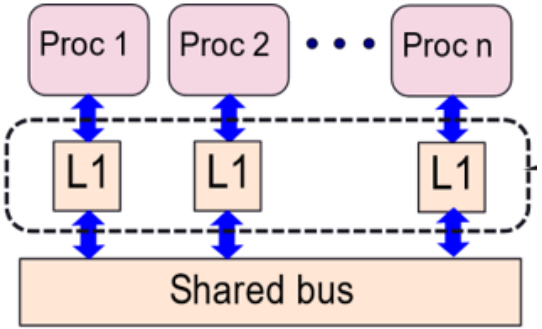
Indian Institute of Technology Guwahati

Symmetric Multiprocessor



Distributed Cache Memories

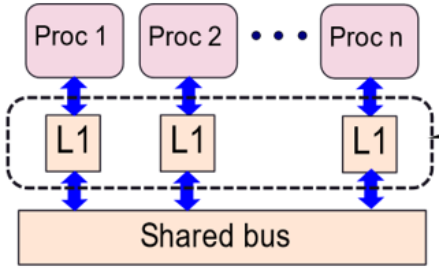




- For multiple processors, caches have to be distributed
- Small simple private L1 caches
- Parallel access
- Can we treat all these individual caches act like one big cache?

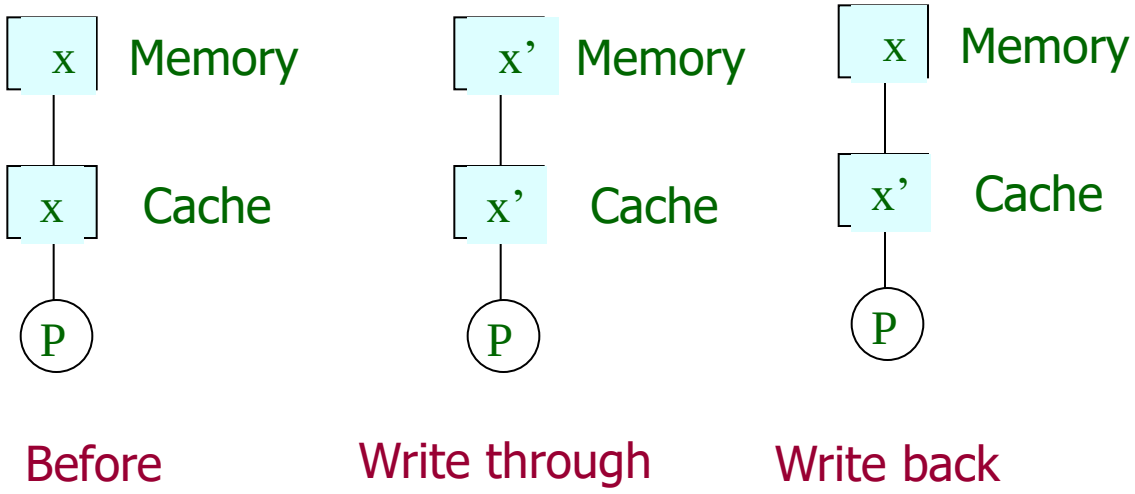
Slides courtesy: Advanced computer Architecture: S.R. Sarangi, IIT Delhi

Problem of Coherence



- Consider access to memory location [variable] X?
- If we have multiple locations for storing the variable x, then the replicas have to contain the same value. [Problem of Coherence]
- **Coherence** is about ordering of operations from different processors to the same memory location

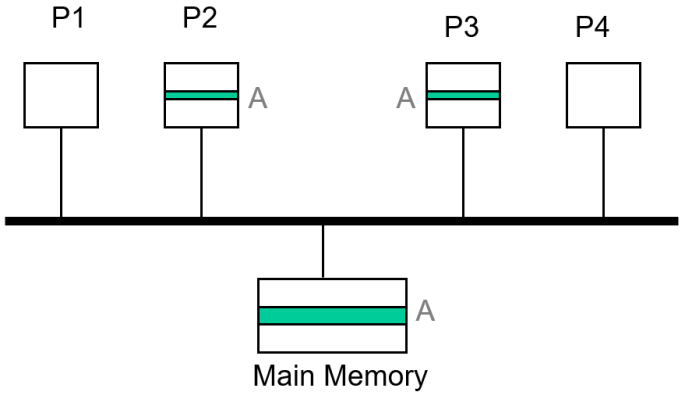
Writing in the cache



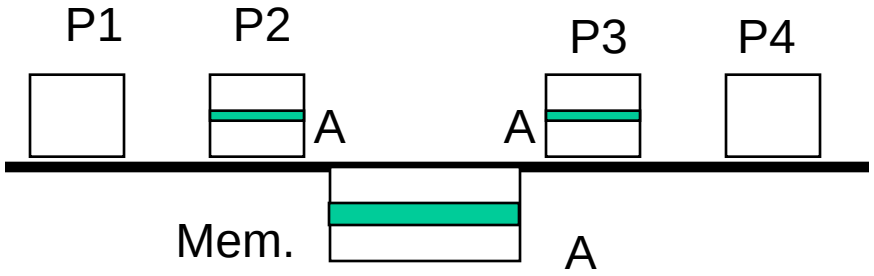
Cache Coherence Problem

❖Consider the follow of events.

1. P2 reads A [Read miss, Fetch A]
2. P3 reads A [Read miss, Fetch A]
3. P2 wants to write A [Write Hit]
4. P3 reads A [Read hit?]
5. How will P3 gets latest value of A?

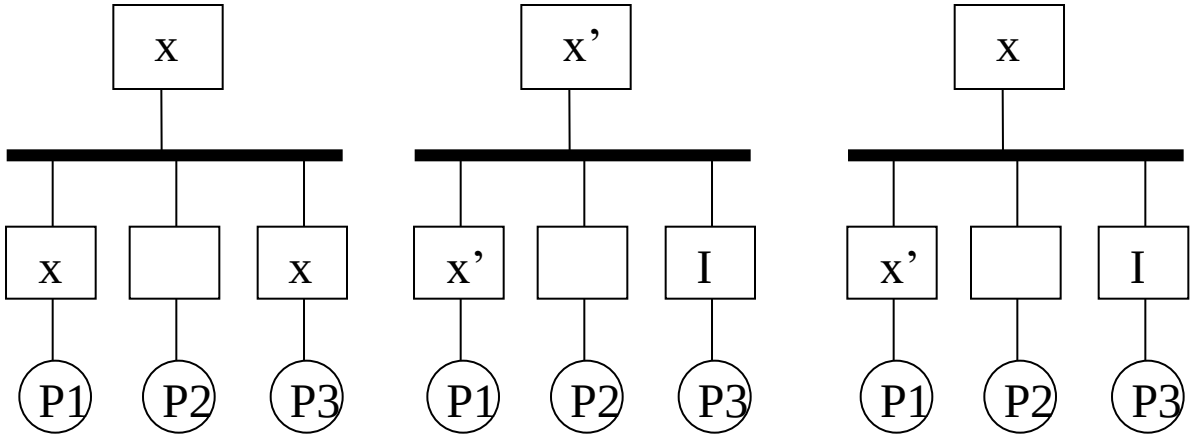


Cache Coherence on shared-bus



- Two choices:
  - **Write-update:** Broadcast the new value of A on the bus by P2; value of A snooped by cache of P3. Memory is also updated.
  - **Write-invalidate:** Broadcast an invalidation message with the address of A; the address snooped by cache of P3 which invalidates its copy of A: The copy in memory is not up-to-date any longer.

Write-Invalidate

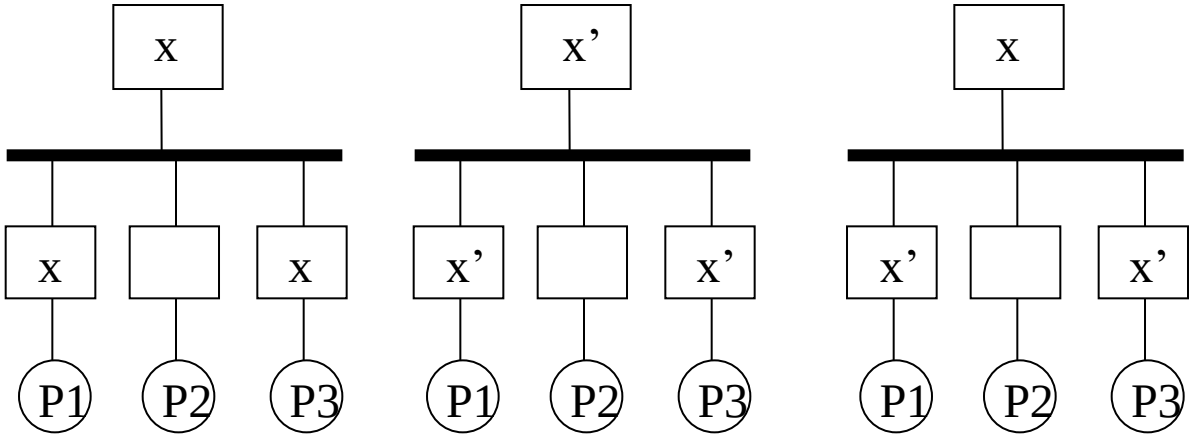


Initial State

Write Through

Write Back

Write-Update



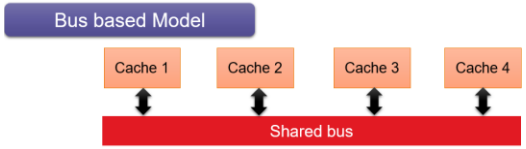
Initial State

Write Through

Write Back

Snooping Protocols

- Protocols that use bus are known as Snoopy Cache Coherence protocols
- All messages are essentially broadcast messages
- All the caches can snoop these broadcasts



- Based on response of a snoop, perform coherency commands
- Each block [in cache and main memory] has a state associated with it, which determines what all operations are to be done to use the block
- The state of a block might change as a result of the operations
  - Read-Miss
  - Write-Miss
  - Write-Hit

Snooping Protocols

- **Write Invalidate**
  - CPU requesting to write to an address, grabs bus, and sends a '**write invalidate**' message
  - All snooping caches invalidate their copy of appropriate cache block
  - CPU writes to its cached copy (Write Through Cache-So writes through to memory also)
  - Any shared read in other CPUs will now miss in cache and re-fetch new data

Snooping Protocols

- **Write Update**
  - CPU requesting to write grabs bus and broadcasts new data as it updates its own copy
  - All snooping caches update their copy
- Problem of simultaneous writes is taken care of by bus arbitration
- Only one CPU can use the bus at a time

Update or Invalidate?

- Update looks the simplest, most obvious and fastest
- Invalidate scheme is usually implemented with write-back
  - Multiple writes to same word (no intervening read) need only one invalidate message but would require an update for each
  - Writes to same block in a multi-word cache block require only one invalidate but would require multiple updates.
- Bus bandwidth is a costly resource in shared memory multi-processors
- Invalidate protocols use significantly less bandwidth.

MESI Protocol

- Invalidate protocol with a write back scheme
- Single Writer & Multiple Reader Model for each Cache Block
- **MESI** → **M**odified **– E**xclusive **– S**hared **- I**nvalid

- Cache block changes state on memory access events
  - Due to local processor activity (i.e. cache access)
  - Due to bus activity - as a result of snooping. Cache block state affected only if address matches

MESI Protocol

- **MESI** → **M**odified **– E**xclusive **– S**hared **- I**nvalid
- A cache block can be in one of 4 states (2 bits)
- **Modified** - cache block has been modified, is different from main memory - is the only cached copy (‘dirty’)
- **Exclusive** - cache block is the same as main memory and is the only cached copy
- **Shared** - Same as main memory but copies may exist in other caches
- **Invalid** - The data is not valid

MESI Protocol

- Operation can be described informally by looking at action in local processor
  - Read Hit
  - Read Miss
  - Write Hit
  - Write Miss

- Defined by a state transition diagram

MESI Local Read Hit

- **Block must be in one of M – E – S**
  - This must be correct local value
  - If M, it must have been modified locally
  - Simply return value
  - No state change

MESI - Local Read Miss

Case 1: No other copy in caches

- ❖ Processor makes bus request to memory
- ❖ Value read to local cache from memory, mark it as E

Case 2: One cache has E copy

- ❖ Processor makes bus request to memory
- ❖ Snooping cache puts copy value on the bus
- ❖ Memory access is abandoned
- ❖ Local processor caches value
- ❖ Both cache blocks set to S

MESI - Local Read Miss

Case 3: Several caches have S copy

- Processor makes bus request to memory
- One cache puts copy value on the bus (arbitrated)
- Memory access is abandoned
- Local processor caches value
- Local copy set to S
- Other copies remain S

MESI - Local Read Miss

Case 4: One cache has M copy

- Processor makes bus request to memory
- Snooping cache puts copy value on the bus
- Memory access is abandoned
- Source (M) value copied back to memory
- Local processor caches value
- Local copy tagged S
- Source value move from M → S

MESI Local Write Hit

Case 1: One cache has M copy

- Block is exclusive and already ‘dirty’
- Update local cache value, no state change

Case 2: One cache has E copy

- Update local cache value, State change from E → M

Case 3: Several caches have S copy

- Processor broadcasts an invalidate on bus
- Snooping processors with S copy, change S→I
- Local cache value is updated and Local state change S→M

MESI Local Write Miss

Case 1: No other copies

- Value read from memory to local cache
- Value updated and local copy state set to M

Case 2: Other copies, either one in state E or more in state S

- Value read from memory to local cache - bus transaction marked RWITM (read with intent to modify)
- Snooping processors see this and set their copy state to I
- Local copy updated and state set to M

MESI Local Write Miss

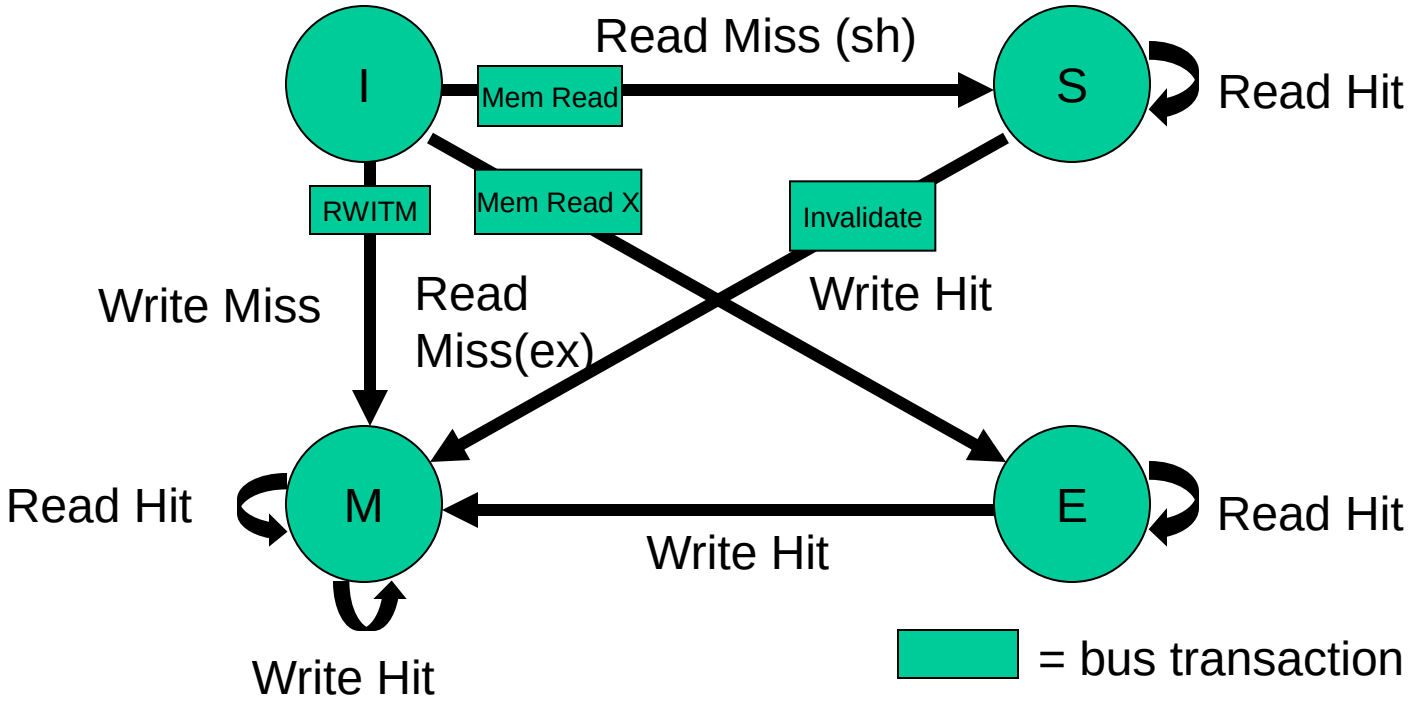
Case 3: Another copy in state M

- Processor issues bus transaction marked RWITM (read with intent to modify)
- Snooping processor sees this
- Blocks RWITM request, takes control of bus
- Writes back its copy to memory, Sets its copy state to I
- Original local processor re-issues RWITM request
- Is now simple no-copy case
- Value read from memory to local cache
- Local copy value updated, Local copy state set to M

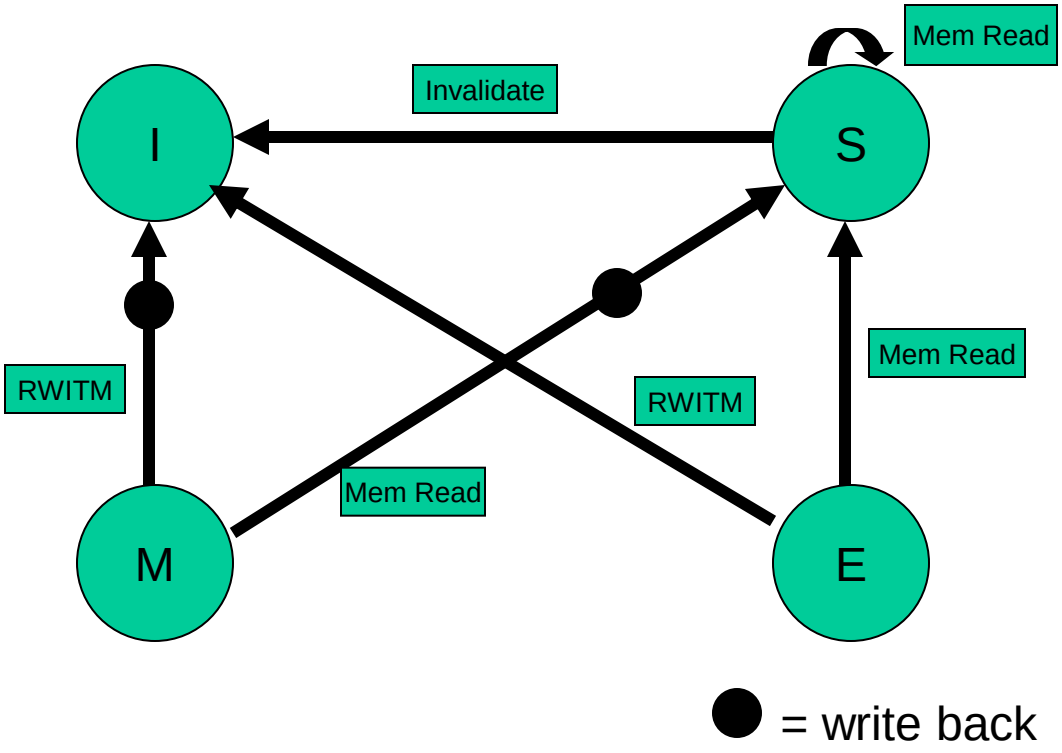
MESI - FSM

- Description of all state operations using a state transition diagram
- Diagram shows what happens to a cache block in a processor
  - memory accesses made by a processor (read hit/miss, write hit/miss)
  - memory accesses made by other processors that result in bus transactions observed by local snoopy cache (Mem read, RWITM, Invalidate)

MESI – locally initiated accesses



MESI – remotely initiated accesses



johnjose@iitg.ac.in
http://www.iitg.ac.in/johnjose/